

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
**Национальный исследовательский ядерный университет «МИФИ»**

---



**Институт  
интеллектуальных кибернетических систем  
Кафедра №22 «Кибернетика»**

**Система автоматизированного развёртывания окружений**

**Стандарт на программирование**

Выполнили:

Кузнецова А. А.

Куренный Г. И.

Левченко А. А.

Сысоев К. В.

Чен П. Я.

Б20-504

Деревянко А.К.

Б20-564

Проверяющий:

Козырев В. П.

# Содержание

|                                                                 |          |
|-----------------------------------------------------------------|----------|
| <b>Стандарт на программирование</b>                             | <b>3</b> |
| 1. Стандарт для разработки backend-составляющей                 | 3        |
| 1.1. Используемый стек технологий                               | 3        |
| 1.2. Используемые инструменты                                   | 3        |
| 1.3. Стандарт оформления кода на языке Go                       | 3        |
| 2. Стандарт для оформления frontend-составляющей                | 7        |
| 2.1. Используемый стек технологий                               | 7        |
| 2.2. Используемые инструменты                                   | 7        |
| 2.3. Структура проекта                                          | 7        |
| 2.4. Стандарт оформления кода на языках JavaScript и TypeScript | 7        |
| 3. Стандарт проектирования базы данных                          | 11       |
| 3.1. Этапы проектирования базы данных                           | 11       |
| 3.1.1. Концептуальное проектирование                            | 11       |
| 3.1.2. Логическое проектирование                                | 13       |
| 3.1.3. Физическое проектирование                                | 14       |
| 3.2. Стандарт оформления скрипта на языке SQL                   | 14       |
| 3.3. Нормализация базы данных                                   | 15       |
| 4. Исключения из правил                                         | 16       |

# Стандарт на программирование

В данном разделе будут описаны те стандарты, которые будут использоваться при дальнейшей разработке системы, в том числе это стандарт на разработку требований, основанный на IEEE830-1998 и стандарт на кодирование и оформление код

## 1. Стандарт для разработки backend-составляющей

### 1.1. Используемый стек технологий

1. Язык разработки - Go, версия - 1.18
2. В качестве протокола передачи данных между backend и frontend частями используется HTTP, в качестве стандарта для передачи данных был выбран REST API

### 1.2. Используемые инструменты

1. IDE - JetBrains Goland 2023.3.1 - среда для разработки на языке Go
2. Линтеры - go vet, staticcheck, errcheck, golint

### 1.3. Стандарт оформления кода на языке Go

Язык Go обладает определенными его разработчиками и сообществом единым сводом правил оформления кода (Effective Go - The Go Programming Language), который будет применяться в данном проекте. Также, за счет описанных выше линтеров будет осуществляться форматирование кода проекта (goimports) и систематизироваться наименования переменных (revive).

Свод правил, используемых для написания кода на языке Go в проекте:

1. Наименование переменных и функций:
  - Использовать camelCase для именования переменных и функций.
  - Названия должны быть короткими и описательными, это означает, что названия должны быть краткими, чтобы улучшить читаемость кода, но в то же время должны содержать достаточно информации, чтобы легко

*// Приемлемо*

*firstName := "John"*

*lastName := "Doe"*

```
fullName := getFullName(firstName, lastName)
```

```
// Неприемлемо
```

```
fName := "John"
```

```
lName := "Doe"
```

```
finalName := getN(fName, lName)
```

2. Форматирование кода:

- Строки кода должны содержать максимум 120 символов.

3. Обработка ошибок:

- Всегда проверяйте возвращаемые функциями значения ошибок. Данный пункт относится только к функциям, которые могут возвращать ошибки
- Избегать обработки ошибок сразу после их обнаружения. Обработка ошибок должна быть в самом верхнем уровне функции.

```
// Приемлемо
```

```
func readData() error {
```

```
    file, err := os.Open("file.txt")
```

```
    if err != nil {
```

```
        // Ошибка обнаружена, но обработка ошибки будет проводиться в вызывающей функции
```

```
        return err
```

```
    }
```

```
    /*
```

```
    */
```

```
}
```

```
func main() {
```

```
    err := readData()
```

```
    if err != nil {
```

```
        // Обработка ошибки на самом верхнем уровне функции
```

```
        log.Fatal(err)
```

```
    }
```

```
}
```

```
// Неприемлемо
```

```
func readData() error {
```

```
    file, err := os.Open("file.txt")
```

```

if err != nil {
    // Обработка ошибки сразу при обнаружении
    log.Fatal(err)
}
/*
*/
}

```

#### 4. Использование пакетов и импортов:

- Импорты должны быть в алфавитном порядке и разделены пустой строкой.

*// Приемлемо*

```

import (
    "fmt"
    "os"
)

```

```

import "fmt"

```

*// Неприемлемо*

```

import ("fmt" "os")

```

#### 5. Комментарии:

- Всегда добавлять комментарии к неочевидным или важным частям кода. Такие места выбираются самим разработчиком, и их выбор может быть обсужден и утвержден ревьюером его кода. Комментарии должны кратко пояснять, что делает определенная часть кода или почему она реализована именно таким образом.
- Не писать комментариев, которые просто повторяют код.

*// Приемлемо*

*// calculateTotalAmount рассчитывает общую сумму для заданного товара и количества.*

```

func calculateTotalAmount(itemPrice float64, quantity int) float64 {
    // ...
}

```

*// Неприемлемо*

*// функция calculateTotalAmount*

```

func calculateTotalAmount(itemPrice float64, quantity int) float64 {

```

```
// ...  
}
```

#### 6. Использование констант:

- Использовать `const` для объявления констант.
- Имена констант должны быть в верхнем регистре с подчеркиванием для разделения слов.

```
// Приемлемо  
const MAX_RETRIES = 3  
  
// Неприемлемо  
const maxMetres = 5
```

#### 7. Избыточная проверка на `nil`:

- Проверять на `nil`, только если это необходимо.

```
// Приемлемо  
func GetValueFromMap(m map[string]string, key string) (string, error) {  
    val, ok := m[key]  
    if !ok {  
        return "", errors.New("Key not found")  
    }  
    return val, nil  
}  
  
// Непримлемо  
func main() {  
    var s *string  
    if s != nil {  
        // Нарушение правила, так как здесь проверяется на nil необоснованно, значение  
        // переменной не было проинициализировано, значит она по умолчанию равна nil  
        fmt.Println(*s)  
    } else {  
        fmt.Println("s is nil")  
    }  
}
```

#### 8. Тестирование:

- Использовать пакет `testing` для написания юнит-тестов.
- Имена тестов должны начинаться с `Test`.

```
// Приемлемо
func TestCalculateTotalAmount(t *testing.T) {
    // ...
}

// Неприемлемо
fun calcualateTotalAmount_test() {}
fun TcalcualateTotalAmount() {}
```

#### 9. Структуры данных:

- Использовать структуры данных для группировки связанных данных.
- Создавать структуры данных только тогда, когда они необходимы для работы нескольких функций.

## 2. Стандарт для оформления frontend-составляющей

### 2.1. Используемый стек технологий

В качестве языков разработки frontend-части приложения будут использоваться JavaScript и TypeScript.

### 2.2. Используемые инструменты

Среда разработки - Sublime Text3

### 2.3. Структура проекта

Проект будет содержать три основных директории:

1. modules - Директория для хранения всех установленных зависимостей проекта.
2. public - Директория, содержащая результат сборки проекта.
3. src - Директория, хранящая исходные файлы проекта.

Помимо этого в корневом папке проекта будут находиться следующие файлы:

1. package.json - файл, отвечающий за хранения версий используемых в проекте frontend-фреймворков и библиотек.
2. vue.config.js - исходный файл конфигурации проекта.

### 2.4. Стандарт оформления кода на языках JavaScript и TypeScript

В качестве стандарта для оформления кода на языках JavaScript и TypeScript будут использоваться модифицированные ESLint-правила. Ниже приведены правила, включенные в модификацию:

## 1. Табуляция кода

- Использовать два пробела в качестве отступа в начале строки.
- Использовать пробел в качестве разделителя между закрывающей круглой скобкой и открывающей фигурной скобкой при описании функции.
- При инициализации значения переменной, знак равенства должен быть окружен единичными пробелами.
- Не допускается добавление пробелов в конце строки.

*// Неприемлемо*

```
function foo(){  
···var bar=1····;  
}
```

*// Приемлемо*

```
function foo(){  
··var bar=1;  
}
```

## 2. Инициализация строк:

- Строки должны быть заключены в двойные кавычки.
- Строки, состоящие из одного символа должны быть заключены в одинарные кавычки.
- Использовать шаблонные строки вместо объединения строк.

*// Неприемлемо*

```
var str = 'Hello, World!';  
var letterA = "a"  
var greeting = "Hello, " + name + "!";
```

*// Приемлемо*

```
var str = "Hello, World!";  
var letterA = 'a';  
var greeting = "Hello, ${name}!";
```

## 3. Завершение строки

- Необходимо всегда ставить точку с запятой в конце строки с объявлением и/или инициализацией переменной, вызове функции.
- Точка с запятой не ставится в конце закрывающей скобки условной конструкции if.

*// Неприемлемо*

```
var x = 5
let z
if (a > b) {
  // code
};
```

*// Приемлемо*

```
var x = 5;
if (a < c) {
  // code
}
```

#### 4. Именованние переменных и функций

- Для наименования переменных и функций необходимо использовать camelCase.
- Для наименования констант необходимо использовать верхний регистр со знаком нижнего подчеркивания для разделения слов.
- Названия должны быть краткими, чтобы обеспечить легкость чтения кода, но содержательными, чтобы был понятен их смысл.

*// Неприемлемо*

```
var u_Name = 'John';
const min = -123899;
```

*// Приемлемо*

```
var userName = 'John';
const MIN = -123904;
```

#### 5. Ключевые слова для создания переменных:

- Использовать const или let для объявления изменяемых переменных.
- Не использовать var

*// Неприемлемо*

```
var x = 5;
```

*// Приемлемо*

```
const x = 5;
```

#### 6. Использование глобальных переменных:

- Не использовать глобальные переменные.

*// Неприемлемо*

```
var globalVar = 10;
```

```
// Приемлемо
```

```
const localVar = 10;
```

#### 7. Использование комментариев:

- Всегда добавлять комментарии к неочевидным или важным частям кода. Такие места выбираются самим разработчиком, и их выбор может быть обсужден и утвержден ревьюером его кода. Комментарии должны кратко пояснять, что делает определенная часть кода или почему она реализована именно таким образом.
- Не писать комментариев, которые просто повторяют код.

```
// Неприемлемо
```

```
// Увеличиваем счетчик на 1
```

```
counter = counter + 1;
```

```
// Приемлемо
```

```
counter++;
```

#### 8. Анонимные и стрелочные функции

- Функции, запись тела которых помещается в одну строку, должны быть анонимными. Если тело функции занимает 2 и более строки, то она должна быть именованной.
- Все анонимные функции должны быть записаны с помощью стрелочных функций.

```
// Неприемлемо
```

```
function(x) {
```

```
    return x * 2;
```

```
}
```

```
function(x) {
```

```
    if (x > 2) {
```

```
        // some code
```

```
    } else {
```

```
        // some code
```

```
    }
```

```
}
```

```
// Приемлемо
```

```
(x) => x * 2;
```

## 9. Сравнение значений и типов:

- Использовать === вместо == для сравнения значений и типов:

*// Неприемлемо*

```
if (x == '10') {
```

```
  // code
```

```
}
```

*// Приемлемо*

```
if (x === '10') {
```

```
  // code
```

```
}
```

## 10. Форматирование кода:

- Длина строки не должна превышать 120 символов.

*// Неприемлемо*

```
const longString = 'Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.';
```

*// Приемлемо*

```
const shortString = 'Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt.';
```

# 3. Стандарт проектирования базы данных

## 3.1. Этапы проектирования базы данных

### 3.1.1. Концептуальное проектирование

Концептуальное проектирование направлено на создание моделей, которые отражают предметную область и удовлетворяют информационные потребности пользователей. В процессе проектирования будет использоваться модель "сущность-связь" (Entity-Relationship) и метод ER-диаграммы для создания модели данных.

Главные элементы ER-модели:

- Объекты или же сущности
- Атрибуты между объектами
- Связи между объектами

Объекты могут относиться к следующим типам:

1. Основная сущность - является независимой сущностью

2. Связанная сущность - связана с несколькими родительскими сущностями и содержит информацию о связях сущностей;
3. Классифицирующая сущность – частный случай связанной сущности, не имеющей собственных атрибутов (только атрибуты родительских сущностей, мигрировавших в качестве внешнего ключа);
4. Описательная сущность – сущность, связанная только с одной родительской и хранящая информацию о характеристиках родительской сущности;
5. Подчиненная сущность – дочерняя сущность в иерархии наследования.

У атрибутов сущностей можно выделить несколько типов:

1. Идентифицирующие атрибуты имеют уникальное значение для экземпляров данного типа сущности, в то время как описательные атрибуты содержат интересующие свойства этой сущности. Каждая сущность обязательно должна иметь первичный ключ. А
2. Составные (состоящие из нескольких компонентов) или простые (состоящими из одного компонента).
3. Однозначные (имеющие одно значение для каждого экземпляра сущности) или многозначные (имеющие много значений для каждого экземпляра сущности).
4. Основные (значение не зависит от других атрибутов) или производные (значение вычисляется на основе значений других атрибутов).
5. Обязательные (значение всегда должно быть указано при добавлении данных в базу данных) или необязательные (значение необязательного атрибута может быть пропущено).

Связи между сущностями характеризуются типом связи и классом принадлежности.

При этом, связи бывают следующих типов:

1. Один-к-одному (1:1) - применяется в тех случаях, когда существует необходимость взаимосвязи между двумя сущностями, при этом каждая сущность имеет только одно соответствующее значение другой сущности. Примером связи один-к-одному может быть связь между сущностями "Студент" и "Паспортные данные". Каждому студенту может соответствовать только одна запись о его паспортных данных, а каждым паспортным данным будет соответствовать только один студент.
2. Один-ко-многим (1:N) - используются в различных ситуациях, где существует необходимость связать одну сущность с несколькими экземплярами другой сущности. Примером связи один-ко-многим может быть связь между

сущностями "Страны" и "Города". Каждой стране может соответствовать несколько городов, но каждый город будет принадлежать только одной стране.

3. Многие-ко-многим (N:M) - позволяет представить сложные и взаимосвязанные отношения между сущностями, где каждая из них может иметь несколько экземпляров другой сущности. Примером связи многие-ко-многим может быть связь между сущностями "Студенты" и "Клубы". Здесь каждый студент может быть членом нескольких клубов, и каждый клуб может иметь несколько студентов-членов.

Результатом концептуального проектирования является создание ER-модели базы данных.

### **3.1.2. Логическое проектирование**

Логическое проектирование направлено на определение необходимого числа и структуры таблиц, создание запросов к базе данных, определение типов отчетов, разработку алгоритмов обработки информации, создание форм для ввода и редактирования данных, а также решение других задач, причем особое внимание уделяется проблеме структуризации данных.

Важно, чтобы проектируемая БД обеспечивала целостность данных. Для этого на этапе логического проектирования необходимо накладывать следующие внутренние ограничения целостности:

- Ограничения на значения ключевых атрибутов. Они определяются уникальностью значений первичных и альтернативных ключей в структуре данных. Подобные ограничения устанавливаются путем выделения ключевых атрибутов в концептуальной модели.
- Ограничения на допустимые значения атрибутов. Указывают, что все значения атрибутов должны соответствовать условиям, заданным для соответствующего домена. Эти ограничения определяются в описании доменов и атрибутов.
- Ограничения на существующие значения. Указывают, что существование некоторых сущностей (например, подкатегорий) зависит от существования других (например, категорий-родителей).

Результатом логического проектирования является логическая модель базы данных. Для проектирования логической модели должна быть использована нотация "Crow's foot".

### **3.1.3 Физическое проектирование**

Цель физического проектирования включает в себя описание метода реализации логического проекта базы данных. На данном этапе, касающемся реляционных баз данных, сущности преобразуются в таблицы, а атрибуты – в столбцы. Взаимосвязи между сущностями трансформируются в физическую схему базы данных.

Также, на данном этапе решается задача оптимального распределения элементов базы данных в памяти, создании индексов, оценке целесообразности применения хеширования и кластеризации, с учетом особенностей выбранной системы управления базами данных.

Результатом физического проектирования является скрипт для создания базы данных на языке SQL.

## **3.2. Стандарт оформления скрипта на языке SQL**

### **1. Ясные и понятные имена таблиц:**

Имена таблиц должны отражать содержание данных без использования слишком абстрактных или коротких названий.

### **2. Избегайте использования зарезервированных слов:**

Имена таблиц и столбцов не совпадают с зарезервированными словами в SQL, это необходимо, чтобы избежать конфликтов.

### **3. Краткость и ясность имен столбцов:**

Имена столбцов должны быть краткими, но информативными. Необходимо избегать использования неясных сокращений.

### **4. Использование первичных ключей:**

Каждая таблица должна иметь первичный ключ, который уникально идентифицирует каждую запись. Это обеспечивает уникальность данных и повышает эффективность запросов.

### **5. Определение внешних ключей:**

Если есть связи между таблицами, то требуется явно определить внешние ключи для поддержания целостности данных.

### **6. Комментарии к таблицам и столбцам:**

К таблицам и столбцам должны быть оставлены подробные комментарии, описывающие их назначение и ожидаемые типы данных.

### **7. Выбор подходящих типов данных:**

Для каждого столбца необходимо выбрать подходящий тип данных, чтобы минимизировать использование памяти и повысить эффективность запросов.

8. Соблюдение нормализации:

Чтобы избежать избыточности данных и обеспечить легкость обновления, необходимо проектировать базу данных с учетом правил нормализации. Правила нормализации и виды нормальных форм описаны в раздела “Нормализация базы данных”.

9. Индексы для ускорения запросов:

Создавать индексы для столбцов, используемых в условиях WHERE или JOIN, для повышения скорости выполнения запросов.

10. Регулярные резервные копии:

Разработать систему регулярных резервных копий базы данных для предотвращения потери данных в случае сбоев.

11. Ограничения целостности данных:

Задавать ограничения целостности данных (например, NOT NULL), чтобы гарантировать корректность данных.

12. Использование символов в именах:

Не использовать пробелы и специальные символы в именах таблиц и столбцов для избежания проблем при написании SQL-запросов.

13. Документация скрипта:

Необходимо предоставить документацию к скрипту, описывающую структуру базы данных, используемые индексы и любые особенности реализации.

### **3.3. Нормализация базы данных**

При разработке реляционных баз данных проводится нормализация, что представляет собой процесс приведения структуры отношений в базе данных к соответствию нормальным формам.

Виды нормальных форм:

- Первая нормальная форма (1НФ). Когда отношение в базе данных соответствует первой нормальной форме, это означает, что каждый кортеж содержит лишь одно значение для каждого из его атрибутов в любом допустимом состоянии этого отношения.
- Вторая нормальная форма (2НФ). Вторая нормальная форма достигается в отношении базы данных, когда оно уже соответствует первой нормальной

форме, и при этом каждый атрибут, не входящий в состав ключа, функционально полностью зависит от всего ключа.

- Третья нормальная форма (3НФ). Третья нормальная форма достигается в отношении базы данных, когда оно уже соответствует второй нормальной форме, и при этом отсутствуют транзитивные функциональные зависимости между неключевыми атрибутами и ключевыми атрибутами.
- Нормальная форма Бойса-Кодда (BCNF) или усиленная третья нормальная форма. Отношение считается находящимся в этой форме, если каждая не тривиальная и неприводимая слева функциональная зависимость в нем имеет потенциальный ключ в качестве своего детерминанта
- Четвертая нормальная форма (4НФ). Данная НФ описывает переменную отношения, находящуюся в этой форме, если она соответствует нормальной форме Бойса — Кодда и не содержит нетривиальных многозначных зависимостей.
- Пятая нормальная форма (5НФ). Переменная отношения считается находящейся в пятой нормальной форме, тогда и только тогда, когда каждая нетривиальная зависимость соединения в ней определена потенциальным ключом (или ключами) этого отношения.
- Доменно-ключевая нормальная форма (ДКНФ). Переменная отношения считается находящейся в ДКНФ, тогда и только тогда, когда каждое наложенное на неё ограничение является логическим следствием ограничений доменов и ограничений ключей, примененных к данной переменной отношения.
- Шестая нормальная форма (6НФ). Переменная отношения считается находящейся в шестой нормальной форме, тогда и только тогда, когда она удовлетворяет всем нетривиальным зависимостям соединения. Согласно данному определению, переменная находится в 6НФ, только если она является неприводимой, то есть не может быть подвергнута дальнейшей декомпозиции без потери информации.

#### **4. Исключения из правил**

Исключения из установленных правил допустимы лишь в случае, если разработчик способен обосновать и объяснить их необходимость.